Lab 7: Fuzzing for Input Validation Bugs

Application: Car Rental Rate Technique: Hypothesis property-based testing

Objective

Generate unexpected text and None inputs for all processor functions, identify failures, and harden the application so malformed input never exposes parsing exceptions.

Figure 1 - Initial fuzz-test failures

```
$ .venv/bin/python -m pytest test_processor.py -q
FFF

FAILED test_sanitize_license_plate_no_crash
  s = None -> AttributeError: NoneType has no attribute strip

FAILED test_parse_trip_mileage_safe
  s = ''   -> ValueError: could not convert string to float
  s = None -> AttributeError: NoneType has no attribute split

FAILED test_format_rental_invoice_safe
  s = ':'  -> ValueError: invalid literal for int
  s = ''   -> IndexError: list index out of range
  s = None -> AttributeError: NoneType has no attribute split

3 failed
```

Hypothesis minimized each failure to a small, reproducible input.

Fuzz-test design

Each test uses @given(st.one_of(st.text(), st.none())) and verifies that its function returns the expected result type without throwing an exception.

Defensive fixes

1. sanitize_license_plate returns an empty string for non-text input.
2. parse_trip_mileage ignores blank, invalid, NaN, and infinite numeric fields.
3. format_rental_invoice validates the complete record and positive integral rental days.

Lab 7: Fixed Application Evidence

Figure 2 - Fixed processor run

```
==== Running sanitize_license_plate ====
Enter license plate with delimiters: Sanitized Plate: KA01AB1234

==== Running parse_trip_mileage ====
Parsed Mileage List: [100.5, 45.2, 80.0]

==== Running format_rental_invoice ====
Rental Invoice: Driver Rohan booked SUV for 3 days: Total $150.00
```

Figure 3 - Fuzz tests after the fix

```
$ .venv/bin/python -m pytest test_processor.py -q
...                                                                      [100%]
3 passed in 0.17s
```

Conclusion

All three completed Hypothesis fuzz tests now pass. The processor safely handles generated Unicode text and None input while retaining the expected valid rental workflow.

Commands used

```
python -m venv .venv
.venv/bin/pip install hypothesis pytest
.venv/bin/python processor.py
.venv/bin/python -m pytest test_processor.py -q
```